
PRACTICAL WEEK 10

1. Investigate inode structures using commands such as ls-i, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

Create a test directory:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ mkdir practical10
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cd practical10
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ echo "This is Practical 10 inode testing." > original.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ ls -li original.txt
52714 original.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

Investigate using stat:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ stat original.txt
  File: original.txt
  size: 36          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 52714         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/vishwa_teja_veerandi)   Gid: ( 1000/vishwa_teja_veerandi)
Access: 2026-10-06 14:19:51.356976216 +0000
Modify: 2026-10-06 14:19:51.356976216 +0000
Change: 2026-10-06 14:19:51.356976216 +0000
 Birth: 2026-10-06 14:19:51.356976216 +0000
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

Create a hard link:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ ln original.txt hardlink.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ ls -li
total 8
52714 -rw-r--r-- 2 vishwa_teja_veerandi vishwa_teja_veerandi 36 Oct  6 14:19 hardlink.txt
52714 -rw-r--r-- 2 vishwa_teja_veerandi vishwa_teja_veerandi 36 Oct  6 14:19 original.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

Modify the hard link:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ echo "Hard link modification." >> hardlink.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ cat original.txt
This is Practical 10 inode testing.
Hard link modification.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

Create a symbolic link:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ ln -s original.txt symlink.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ ls -li
total 8
52714 -rw-r--r-- 2 vishwa_teja_veerandi vishwa_teja_veerandi 60 Oct  6 14:22 hardlink.txt
52714 -rw-r--r-- 2 vishwa_teja_veerandi vishwa_teja_veerandi 60 Oct  6 14:22 original.txt
52715 lrwxrwxrwx 1 vishwa_teja_veerandi vishwa_teja_veerandi 12 Oct  6 14:23 symlink.txt -> original.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

Investigate the symbolic link:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ stat symlink.txt
  File: 'symlink.txt' -> 'original.txt'
  size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 52715         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/vishwa_teja_veerandi)   Gid: ( 1000/vishwa_teja_veerandi)
Access: 2026-10-06 14:23:50.572968833 +0000
Modify: 2026-10-06 14:23:46.512969215 +0000
Change: 2026-10-06 14:23:46.512969215 +0000
 Birth: 2026-10-06 14:23:46.512969215 +0000
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ stat -L symlink.txt
  File: symlink.txt
  size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 52714         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/vishwa_teja_veerandi)   Gid: ( 1000/vishwa_teja_veerandi)
Access: 2026-10-06 14:22:50.320971503 +0000
Modify: 2026-10-06 14:22:44.816971905 +0000
Change: 2026-10-06 14:22:44.816971905 +0000
 Birth: 2026-10-06 14:19:51.356976216 +0000
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

Use find to investigate inodes:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ find . -inum $(ls -li original.txt | awk '{print $1}')
./original.txt
./hardlink.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ find . -type l
./symlink.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~/practical10$ |
```

2.Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity. same i have to do in ubuntu. this is practical 10.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical10.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>

#define BUFFER_SIZE 4096

double get_time()
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1000000000.0;
}

int copy_read_write(const char *source, const char *destination)
{
    int src = open(source, O_RDONLY);
    if (src < 0)
    {
        perror("open source");
        return -1;
    }

    int dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest < 0)
    {
        perror("open destination");
        close(src);
        return -1;
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        if (write(dest, buffer, bytes_read) != bytes_read)
        {
            perror("write");
            close(src);
            close(dest);
            return -1;
        }
    }
}
```

```

    close(src);
    close(dest);

    return 0;
}

int copy_mmap(const char *source, const char *destination)
{
    int src = open(source, O_RDONLY);
    if (src < 0)
    {
        perror("open source");
        return -1;
    }

    struct stat st;

    if (fstat(src, &st) < 0)
    {
        perror("fstat");
        close(src);
        return -1;
    }

    size_t size = st.st_size;

    if (size == 0)
    {
        int dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
        close(dest);
        close(src);
        return 0;
    }

    void *src_map = mmap(NULL, size, PROT_READ, MAP_PRIVATE, src, 0);

    if (src_map == MAP_FAILED)
    {
        perror("mmap source");
        close(src);
        return -1;
    }

    int dest = open(destination, O_RDWR | O_CREAT | O_TRUNC, 0644);

    if (dest < 0)
    {
        perror("open destination");
        munmap(src_map, size);
    }
}

```

```

        close(src);
        return -1;
    }

    if (ftruncate(dest, size) < 0)
    {
        perror("ftruncate");
        munmap(src_map, size);
        close(src);
        close(dest);
        return -1;
    }

    void *dest_map = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, dest, 0);

    if (dest_map == MAP_FAILED)
    {
        perror("mmap destination");
        munmap(src_map, size);
        close(src);
        close(dest);
        return -1;
    }

    memcpy(dest_map, src_map, size);

    msync(dest_map, size, MS_SYNC);

    munmap(src_map, size);
    munmap(dest_map, size);

    close(src);
    close(dest);

    return 0;
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <source_file>\n", argv[0]);
        return 1;
    }

    const char *source = argv[1];

    double start = get_time();

```

```

    if (copy_read_write(source, "copy_read_write.txt") != 0)
        return 1;

    double end = get_time();

    double read_write_time = end - start;

    start = get_time();

    if (copy_mmap(source, "copy_mmap.txt") != 0)
        return 1;

    end = get_time();

    double mmap_time = end - start;

    printf("Traditional read/write time: %.6f seconds\n", read_write_time);
    printf("Memory-mapped I/O time: %.6f seconds\n", mmap_time);

    printf("\nFiles created:\n");
    printf("copy_read_write.txt\n");
    printf("copy_mmap.txt\n");

    return 0;
}

```

Create a test file:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical10.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical10.2.c -o practical10.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ echo "Testing memory mapped file I/O in Ubuntu." > mmap_source.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat mmap_source.txt
Testing memory mapped file I/O in Ubuntu.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical10.2 mmap_source.txt
Traditional read/write time: 0.000202 seconds
Memory-mapped I/O time: 0.011536 seconds

Files created:
copy_read_write.txt
copy_mmap.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Verify both files:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat copy_read_write.txt
Testing memory mapped file I/O in Ubuntu.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat copy_mmap.txt
Testing memory mapped file I/O in Ubuntu.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Test with a larger file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ dd if=/dev/zero of=largefile.bin bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.074913 s, 1.4 GB/s
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical10.2 largefile.bin
Traditional read/write time: 0.102969 seconds
Memory-mapped I/O time: 0.093127 seconds

Files created:
copy_read_write.txt
copy_mmap.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```